

ELEN90098 REINFORCEMENT LEARNING FOR ENGINEERING

WORKSHOP 2: Q LEARNING

SEMESTER 2, 2025

This workshop introduces you to the Q Learning algorithm. You will investigate the characteristics of these algorithms and evaluate their performance.

Submission: Each group must upload 2 files:

- (1) A pdf (**.pdf**) file containing your answers to Task 1. Note that:
 - Only one member of the group needs to upload this file;
 - The naming convention of the file is **Workshop2-<LastName1>-<LastName2>.pdf** for a group whose members have last names, LastName1 and LastName2.
- (2) A zip (**.zip**) file compressing your Q Learning code, with the (TODO) blanks all filled. Note that:
 - Only one member of the group needs to upload this file;
 - The naming convention of the file is **Workshop2-Code-<LastName1>-<LastName2>.zip** for a group whose members have last names, LastName1 and LastName2.

Note: Task 1 needs to be answered in this week's submission. The Final Report should include answers for both Task 1 and Task 2.

1. INTRODUCTION

Reinforcement Learning trains agents to make decisions by interacting with an environment. A fundamental algorithm in RL is **Q Learning**, a model-free, value-based method. Q Learning learns an action-value function, or *Q function*, $Q(s, a)$, which estimates the expected return for taking action a in state s and following the optimal policy thereafter. By updating this function through exploration and exploitation, the agent gradually learns an effective policy without requiring knowledge of the environment's dynamics. This makes Q Learning an ideal starting point for understanding RL fundamentals.

We will test the algorithms on the **CliffWalking** environment. **CliffWalking** is a simple grid-world navigation task with negative rewards for falling off the cliff. Up next, Section 2 gives an overview of the environments and a review of Q Learning. Section 3 is the tasks to be done this week.

2. REVIEW

2.1. Environment: Cliff Walking. Cross a gridworld from start to goal while avoiding falling off a cliff ([Gymnasium website](#))

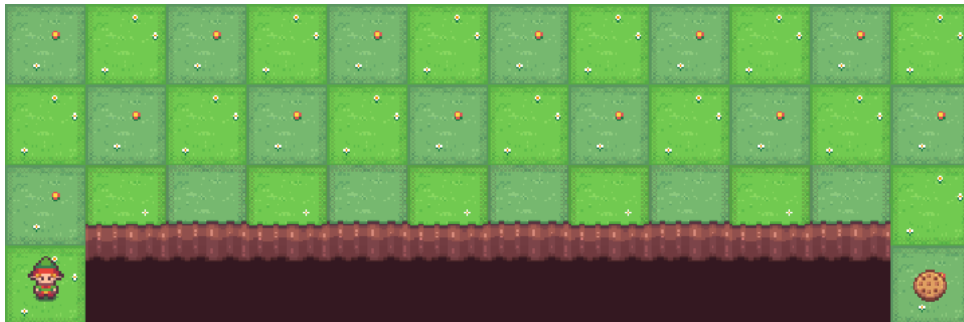


FIGURE 1. CliffWalking-v1

This game starts with the player at location $[3, 0]$ of the 4×12 grid world with the goal located at $[3, 11]$. If the player reaches the goal, the episode ends.

A cliff runs along $[3, 1 : 10]$. If the player moves to a cliff location, it returns to the start location.

The player makes moves until they reach the goal.

Action Space

The action shape is $(1,)$ in the range $0, 3$, indicating which direction to move the player.

- 0: Move up
- 1: Move right
- 2: Move down
- 3: Move left

Observation Space

There are $3 \times 12 + 1$ possible states. The player cannot be on the cliff, nor at the goal, as the latter results in the end of the episode. What remains are all the positions of the first 3 rows plus the bottom-left cell.

The observation is a value representing the player's current position as $current_row \times ncol + current_col$ (where both the row and col start at 0). The observation is returned as an `int()`.

The starting position is $[36]$ (location $[3, 0]$, $3 \times 12 + 0 = 36$).

Rewards

Each time step incurs -1 reward, unless the player steps onto the cliff, which incurs -100 reward.

Termination

The episode terminates when the player enters state $[47]$ (location $[3, 11]$).

2.2. Q Learning and SARSA Variants. We study value-based methods for control, where s is the state, a the action, r the reward, and $Q(s, a)$ the action-value function. Q-learning and SARSA(0), SARSA(n), SARSA(λ) (on-policy) differ in how they update $Q(s, a)$ using the next action or multiple future steps.

Q Learning is an off-policy algorithm that learns the optimal action-value function $Q(s, a)$. At each state s , it updates values toward the greedy action $\arg \max_{a'} Q(s', a')$, aiming directly for the optimal policy π^* .

SARSA(0) (State–Action–Reward–State–Action) is an on-policy algorithm that updates $Q(s, a)$ using the next action a' actually chosen under the current policy π . This often leads to safer, more conservative learning.

SARSA(n) generalizes SARSA(0) by using n -step returns, where updates depend on rewards collected over n future steps.

SARSA(λ) combines all n -step returns through eligibility traces. The parameter $\lambda \in [0, 1]$ controls the weighting of short- vs. long-term returns, providing a trade-off between bias and variance.

3. TASKS

File overview and Implementation. The starter code is provided with blanks marked as (TODO) for you to complete in order to run the algorithm. Move the code under the workshop folder `rl-workshop` that you have created.

The files are organized as follows:

- Q_Learning/
 - `run_q_learning.py`
 - `q_learning/`
 - * `agent.py`
 - `utils/`
 - * `log_utils.py`
 - * `logger.py`
 - * `util.py`

Your work is to complete the (TODO) blanks and implement Q Learning. Before running, navigate to the workshop folder:

```
cd path/to/rl-workshop
```

The starter code automatically logs results to TensorBoard. To visualize training curves, run:

```
tensorboard --logdir ./logs
```

Then open `localhost:6006` in your browser.

—

Task 1: CliffWalking (7 points). In this task, you will explore classical temporal-difference (TD) control methods in reinforcement learning. Specifically, you will compare the performance of Q Learning (off-policy) with SARSA (on-policy) in its different forms:

- SARSA(0) (one-step TD, the standard SARSA algorithm),
- SARSA(n) (n-step TD), and
- SARSA(λ) (TD(λ) with eligibility traces).

You will apply these algorithms to **CliffWalking** and analyze their learning behaviors.

Question 1 (2 points). What are the state space, action space, and reward structure of the **CliffWalking** environment? Are they finite or infinite? Discrete or continuous?

Question 2 (5 points). For the **CliffWalking** environment, run the following algorithms:

- Q Learning (off-policy)
- SARSA(0) (one-step TD)
- SARSA(n) (n-step TD)
- SARSA(λ) (Backward view SARSA(λ) with eligibility traces)

Use TensorBoard to visualize and compare their return curves. Submit a figure showing the learning performance of all four algorithms.

Commands:

Q Learning:

```
python Q_Learning/run_q_learning.py --env_name CliffWalking-v1 --log_video --off_policy
```

SARSA(0):

```
python Q_Learning/run_q_learning.py --env_name CliffWalking-v1 --log_video
```

SARSA(n):

```
python Q_Learning/run_q_learning.py --env_name CliffWalking-v1 --log_video --  
look_ahead_step 5 --exp_name step5
```

SARSA(λ):

```
python Q_Learning/run_q_learning.py --env_name CliffWalking-v1 --log_video --  
sarsa_lambda 0.1 --exp_name lambda0.1
```

—

Task 2. Q-Learning and SARSA

Question 1. Compare the performance of the four algorithms (Q-Learning, SARSA(0), SARSA(n), SARSA(λ)) in terms of both convergence speed and final performance. Discuss possible reasons for the differences you observe.

Question 2. Experiment with different learning rates (α) for Q-learning and compare their effects on performance. Explain why changing α leads to the observed differences.

Question 3. Experiment with different λ for SARSA(λ) and compare their effects on performance. Explain why changing λ leads to the observed differences.

—

Note: Task 1 must be answered this week. Task 2 must be answered in the final report. Answers to all tasks **must be included and discussed in your final report.**